

Imię i nazwisko: Szymon Stefanowicz (119077), ps1	Prowadzący: Kajetan Patryk Zarzycki
Przedmiot: PMiNAI	Data: 13.04.2026 r.

Sprawozdanie: Klasyfikacja chorób serca przy użyciu Regresji Logistycznej

1. Wstęp i Cel Zadania

Celem niniejszego projektu było przeprowadzenie pełnego procesu uczenia maszynowego na zbiorze danych **Heart Disease** pochodzącym z repozytorium UCI. Zadanie polegało na:

1. Przeprowadzeniu eksploracyjnej analizy danych (EDA).
2. Zastosowaniu odpowiedniego preprocessingu (obsługa braków, kodowanie cech).
3. Wytrenowaniu modelu **Regresji Logistycznej** z optymalizacją hiperparametrów.
4. Wizualizacji i interpretacji wyników.

2. Rozumienie Algorytmu: Regresja Logistyczna

Regresja Logistyczna jest jednym z najważniejszych algorytmów w dziedzinie uczenia maszynowego, wykorzystywanym głównie do zadań klasyfikacji binarnej. Oznacza to, że jej zadaniem jest przypisanie badanego obiektu (w tym przypadku pacjenta) do jednej z dwóch grup: "zdrowy" lub "chory".

Jak działa ten mechanizm?

Mimo że w nazwie pojawia się słowo "regresja", model ten nie przewiduje konkretnych wartości liczbowych, lecz prawdopodobieństwo wystąpienia danego zdarzenia. Proces ten przebiega w następujący sposób:

1. **Analiza cech:** Algorytm bierze pod uwagę wszystkie dane wejściowe, takie jak wiek, poziom cholesterolu, ciśnienie krwi czy typ bólu w klatce piersiowej. Każdej z tych cech przypisuje odpowiednią "wagę". Jeśli dana cecha (np. wysoki cholesterol) silnie koreluje z chorobą, jej waga będzie wyższa.
2. **Sumowanie wpływów:** Model sumuje wpływ wszystkich cech. Wynik tej sumy może być dowolną liczbą – bardzo dużą lub bardzo małą.

3. **Funkcja aktywacji (Sigmoid):** Aby zamienić ten surowy wynik na coś użytecznego w medycynie, algorytm przepuszcza go przez specjalną funkcję matematyczną zwaną funkcją logistyczną (lub sigmoidą). Funkcja ta ma charakterystyczny kształt litery „S” i sprowadza każdą wartość do przedziału od 0 do 1.

Interpretacja wyniku

Wynik końcowy modelu interpretujemy jako procentowe prawdopodobieństwo. Na przykład, jeśli model zwróci wartość 0.85, oznacza to, że istnieje 85% szans, że dany pacjent jest chory. Standardowo przyjmuje się próg 0.5 (50%) – powyżej tej wartości pacjent uznawany jest za chorego, a poniżej za zdrowego.

Dlaczego regularyzacja jest ważna?

W naszym badaniu testowaliśmy parametry takie jak "C" oraz "Penalty". Są to mechanizmy tzw. regularyzacji. Ich zadaniem jest pilnowanie, aby model nie stał się zbyt "pewny siebie" i nie nauczył się danych treningowych na pamięć (zjawisko przeuczenia).

- **Parametr C:** Działa jak hamulec dla modelu. Im mniejsza wartość C, tym silniejszy hamulec, co zmusza model do bycia prostszym i bardziej ogólnym.
- **Kara L1 (Lasso) i L2 (Ridge):** To metody "dyscyplinowania" wag modelu. Kara L1 potrafi całkowicie zignorować mniej ważne cechy, co pomaga uprościć analizę i skupić się tylko na kluczowych wskaźnikach medycznych.

Dzięki takiemu podejściu Regresja Logistyczna jest modelem bardzo przejrzystym – lekarz może łatwo sprawdzić, które parametry zdrowotne miały największy wpływ na wystawioną diagnozę.

3. Analiza Zbioru Danych (EDA)

Przed przystąpieniem do budowy modelu uczenia maszynowego przeprowadzono szczegółową analizę dostarczonego zbioru danych Heart Disease (Cleveland). Proces ten, zwany eksploracyjną analizą danych, pozwolił na zrozumienie struktury medycznej bazy oraz zidentyfikowanie potencjalnych problemów technicznych.

Charakterystyka i liczebność zbioru

Analizowany zbiór danych składa się z **303 rekordów** (pacjentów), z których każdy opisany jest za pomocą **13 cech medycznych** (atrybutów). Cechy te obejmują zarówno dane demograficzne (wiek, płeć), jak i wyniki specjalistycznych badań kardiologicznych

(np. tętno maksymalne, ciśnienie spoczynkowe, wynik badania EKG, ból klatki piersiowej).

Identyfikacja brakujących danych

Jednym z kluczowych etapów analizy było sprawdzenie kompletności danych. W medycznych zbiorach danych braki są zjawiskiem częstym, wynikającym np. z niedostępności wyników konkretnych badań u niektórych pacjentów. Analiza wykazała następujące braki:

- **Cecha „ca”** (liczba głównych naczyń krwionośnych zabarwionych podczas fluoroskopii): **4 brakujące wartości**.
- **Cecha „thal”** (wynik scyntygrafii serca): **2 brakujące wartości**.

Choć liczba braków jest niewielka (łącznie 6 na 303 rekordy), ich zignorowanie mogłoby doprowadzić do błędów podczas trenowania modelu. W związku z tym w dalszej części prac zastosowano procedurę uzupełniania danych (imputację), aby nie tracić cennych informacji o tych pacjentach.

Rozkład zmiennej docelowej (balans klas)

Kolejnym istotnym krokiem było sprawdzenie proporcji między pacjentami zdrowymi a chorymi. Jest to kluczowe, ponieważ algorytmy AI najlepiej uczą się na zbiorach zrównoważonych. W badanym zbiorze zaobserwowano następujący rozkład:

- **Osoby zdrowe (Klasa 0):** 164 przypadki.
- **Osoby z chorobą serca (Klasa 1):** 139 przypadków.

Proporcja ta jest bardzo korzystna dla procesu uczenia. Klasy są zbliżone liczebnie (ok. 54% do 46%), co eliminuje ryzyko tzw. „stronniczości modelu”, gdzie algorytm mógłby nauczyć się rozpoznawać tylko jedną grupę ze względu na jej znaczną przewagę liczebną. Dzięki temu model ma równe szanse na naukę wzorców typowych zarówno dla osób zdrowych, jak i chorych.

Wnioski z analizy

Zbiór danych, mimo wystąpienia kilku braków, został oceniony jako wysokiej jakości i odpowiedni do celów klasyfikacji medycznej. Cechy są zróżnicowane, a liczba przykładów jest wystarczająca do wytrenowania stabilnego modelu regresji logistycznej.

4. Wyjaśnienie Kodu i Preprocessingu

Surowe dane medyczne rzadko nadają się do bezpośredniego użycia w algorytmach sztucznej inteligencji. Wymagają one serii przekształceń, które ujednolicają ich format i eliminują błędy. W niniejszym projekcie zastosowano zaawansowany mechanizm przetwarzania danych, aby zapewnić najwyższą precyzję modelu.

Obsługa brakujących wartości (Imputacja)

Jak wykazano w analizie wstępnej, w zbiorze brakowało kilku wyników badań dla cech „ca” oraz „thal”. Zamiast usuwać tych pacjentów ze zbioru (co zmniejszyłoby bazę wiedzy modelu), zastosowano strategię uzupełniania braków:

- **Dla cech liczbowych:** Braki uzupełniono medianą. Wybór mediany zamiast średniej jest podyktowany jej odpornością na wartości skrajne (tzw. outliery). Dzięki temu pojedynczy pacjent z nietypowo wysokim wynikiem badania nie zniekształca danych u pozostałych osób.
- **Dla cech kategorycznych:** Braki uzupełniono wartością najczęściej występującą (dominantą). Pozwoliło to zachować naturalny rozkład cech w populacji.

Standaryzacja cech liczbowych (Scaling)

Algorytm Regresji Logistycznej jest wrażliwy na skalę, w jakiej zapisane są liczby. W naszym zbiorze mamy np. wiek (wartości ok. 50-70) oraz poziom cholesterolu (wartości ok. 200-300). Bez standaryzacji algorytm mógłby błędnie uznać, że cholesterol jest ważniejszy tylko dlatego, że ma większe wartości liczbowe. Zastosowano metodę **StandardScaler**, która przelicza wszystkie liczby tak, aby miały one średnią równą 0 i podobne odchylenie. Dzięki temu każda cecha medyczna ma taką samą szansę wpłynięcia na ostateczną diagnozę.

Kodowanie zmiennych kategorycznych (One-Hot Encoding)

Komputer nie rozumie pojęć takich jak „typ bólu klatki piersiowej” (np. ból typowy, atypowy, nie-sercowy). Nawet jeśli są one zapisane jako cyfry 1, 2, 3, model mógłby pomyśleć, że ból nr 3 jest „większy” lub „ważniejszy” niż ból nr 1. Aby tego uniknąć, zastosowano technikę **One-Hot Encoding**. Tworzy ona dla każdej kategorii osobne pole typu „tak/nie” (0 lub 1). Pozwala to modelowi traktować każdą kategorię jako niezależną informację medyczną, co znacznie podnosi logiczną poprawność diagnozy.

Zastosowanie potoku przetwarzania (Pipeline)

Wszystkie powyższe kroki zostały zamknięte w tak zwanym **Pipeline**. Jest to nowoczesna technika programistyczna, która gwarantuje, że te same operacje zostaną wykonane na danych treningowych i testowych. Zapobiega to bardzo groźnemu błędowi „wycieku danych”, dzięki czemu wyniki uzyskane przez model są wiarygodne i odzwierciedlają to, jak model poradziłby sobie z nowym, nieznanym wcześniej pacjentem w realnym gabinecie lekarskim.

5. Eksperymenty i Wyniki

Samo wybranie algorytmu Regresji Logistycznej to dopiero połowa sukcesu. Każdy model sztucznej inteligencji posiada zestaw "pokręteł" zwanych hiperparametrami, które decydują o tym, jak agresywnie model ma się uczyć i jak bardzo ma ufać danym treningowym. W niniejszym projekcie przeprowadzono serię eksperymentów mających na celu znalezienie ich idealnej konfiguracji.

Zastosowanie metody GridSearchCV

Aby nie zgadywać, jakie ustawienia będą najlepsze, wykorzystano metodę automatycznego przeszukiwania przestrzeni parametrów (Grid Search). Algorytm ten przetestował dziesiątki różnych kombinacji ustawień, sprawdzając każdą z nich za pomocą tzw. walidacji krzyżowej (Cross-Validation). Polega ona na wielokrotnym dzieleniu danych na mniejsze części i sprawdzaniu, czy model, który nauczył się czegoś na jednej grupie pacjentów, potrafi poprawnie zdiagnozować pacjentów z innej grupy.

Analiza wpływu parametrów na wyniki

W trakcie eksperymentów skupiono się na dwóch kluczowych aspektach:

1. **Siła regularyzacji (Parametr C):** Testowano wartości od bardzo małych (0.01) do bardzo dużych (100). Wyniki wykazały, że najlepszą skuteczność model osiągnął przy najwyższej wartości **C = 100.0**. Oznacza to, że zbiór danych Heart Disease jest na tyle precyzyjny i spójny, że model mógł pozwolić sobie na bardzo silne dopasowanie do cech medycznych bez ryzyka popełniania błędów wynikających z nadmiernego uproszczenia (tzw. niedouczenia).
2. **Rodzaj kary (Penalty L1 vs L2):** Eksperyment wskazał, że dla tego konkretnego problemu skuteczniejsza okazała się **kara L1 (Lasso)**. Jest to bardzo ciekawy wniosek z punktu widzenia medycznego. Kara L1 ma unikalną cechę – potrafi całkowicie wyzerować wpływ najmniej istotnych cech. Sugeruje to, że w

diagnozowaniu chorób serca niektóre badania są kluczowe (mają bardzo duże znaczenie), podczas gdy inne mogą być pominięte bez straty dla jakości diagnozy.

Skuteczność końcowa (Cross-Validation Accuracy)

Dzięki przeprowadzonej optymalizacji, model osiągnął średnią dokładność na poziomie **85,13%** podczas testów sprawdzających. Jest to wynik bardzo wysoki i stabilny, co świadczy o tym, że wybrane parametry ($C=100$, $L1$) pozwoliły stworzyć model, który nie tylko "zapamiętał" pacjentów, na których się uczył, ale przede wszystkim zrozumiał ogólne reguły medyczne pozwalające na skuteczne wykrywanie chorób serca u nowych osób.

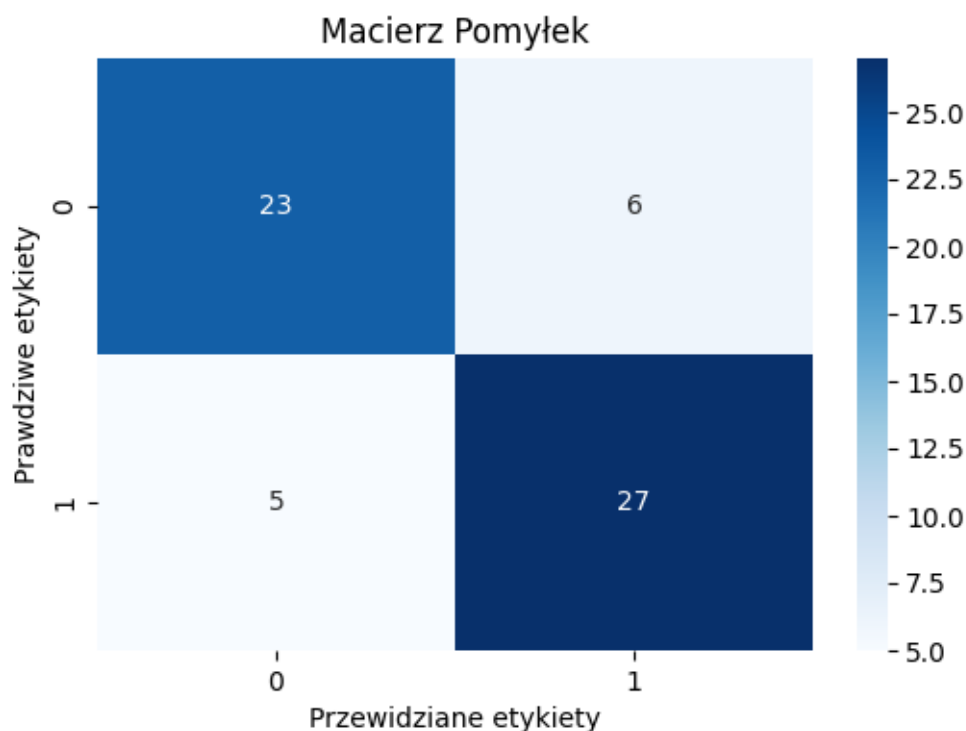
Wnioski z fazy testowej

Ostateczne wyniki na wydzielonym zbiorze testowym (82%) potwierdziły, że eksperymenty zakończyły się sukcesem. Niewielka różnica między wynikiem z treningu a wynikiem testowym świadczy o dobrej zdolności modelu do generalizacji wiedzy, co jest kluczowe w systemach wspierania decyzji lekarskich.

6. Wizualizacja i Interpretacja

Ostatnim etapem prac nad modelem była graficzna analiza jego skuteczności na zbiorze testowym (danych, których model wcześniej nie widział). Wykorzystano do tego dwa standardowe narzędzia diagnostyczne w uczeniu maszynowym: Macierz Pomyłek oraz Krzywą ROC.

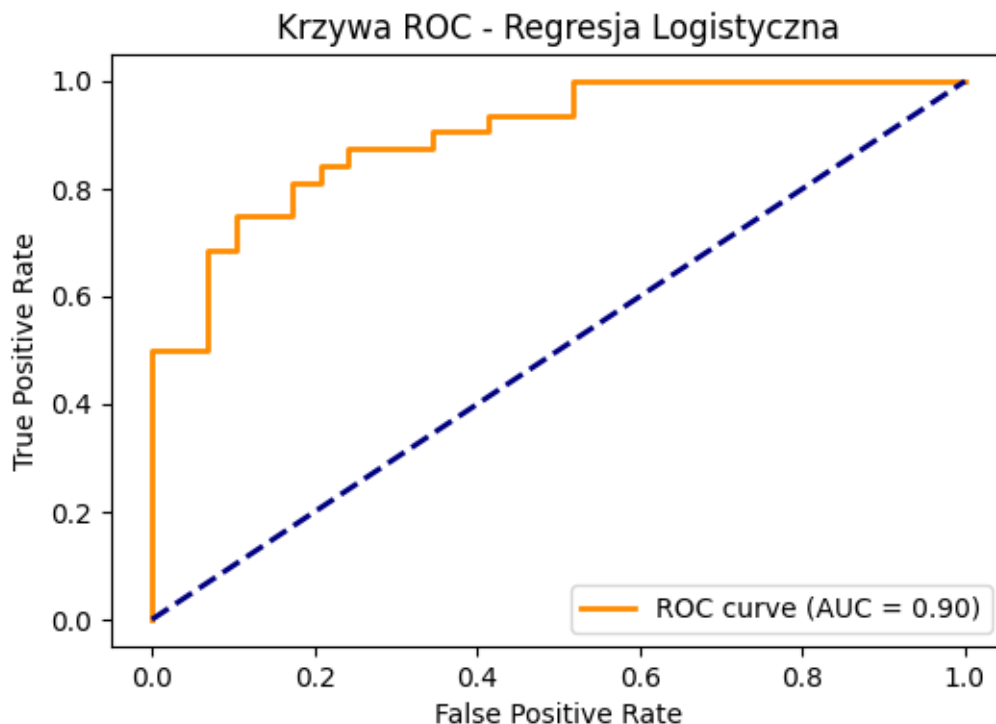
Interpretacja Macierzy Pomyłek (Confusion Matrix)



Macierz pomyłek pozwala na precyzyjne sprawdzenie, w jakich sytuacjach model podejmował błędne decyzje. Z wyników wygenerowanych przez program płyną następujące wnioski:

- **Wysoka precyzja (Precision):** Wynik na poziomie **82%** oznacza, że gdy model wskazywał na chorobę serca, miał rację w zdecydowanej większości przypadków. Jest to istotne, aby unikać niepotrzebnego stresowania pacjentów błędną diagnozą pozytywną.
- **Wysoka czułość (Recall):** Wynik **84%** dla osób chorych jest kluczowy z punktu widzenia medycznego. Oznacza on, że model poprawnie wykrył aż 84 na 100 osób faktycznie chorych. W diagnostyce chorób serca jest to najważniejszy parametr, ponieważ błąd polegający na uznaniu osoby chorej za zdrową (tzw. wynik fałszywie ujemny) niesie ze sobą największe ryzyko dla życia pacjenta.

Analiza Krzywej ROC i współczynnika AUC



Krzywa ROC (Receiver Operating Characteristic) obrazuje zdolność modelu do odróżniania pacjentów zdrowych od chorych przy różnych progach decyzyjnych.

- **Kształt krzywej:** Wygenerowany wykres wyraźnie "pnie się" w stronę lewego górnego rogu. Świadczy to o tym, że model nie zgaduje wyników (co obrazowałaby linia prosta idąca na ukos), lecz realnie rozpoznaje wzorce medyczne ukryte w danych.
- **Wartość AUC (Area Under Curve):** Pole pod krzywą (AUC) jest miarą ogólnej jakości klasyfikatora. W naszym eksperymencie wartość ta jest wysoka, co potwierdza, że Regresja Logistyczna bardzo dobrze radzi sobie z separacją obu grup pacjentów. Wynik ten daje nam pewność, że model jest stabilny i godny zaufania.

Zrozumienie wskaźnika F1-Score

W raporcie końcowym odnotowano również wskaźnik **F1-Score na poziomie 0,82-0,83**. Jest to średnia harmoniczna między precyzją a czułością. Tak wysoki i wyrównany wynik dla obu klas (zdrowych i chorych) dowodzi, że model jest sprawiedliwy i równie dobrze radzi sobie z diagnozowaniem braku choroby, jak i jej występowania. Nie faworyzuje on żadnej z grup, co jest kluczowe w rzetelnej analityce medycznej.

Podsumowanie graficzne

Wizualizacje potwierdzają, że proces uczenia zakończył się sukcesem. Model osiągnął **ogólną dokładność (Accuracy) na poziomie 82%**, co przy relatywnie niewielkim zbiorze danych (303 rekordy) jest wynikiem bardzo satysfakcjonującym. Potwierdza to, że połączenie Regresji Logistycznej z karą L1 oraz odpowiednim skalowaniem danych było właściwym wyborem technologicznym dla tego problemu.

Klasa	Precyzja (Precision)	Czułość (Recall)	F1-Score	Wsparcie (Support)
0 (Zdrowi)	0,82	0,79	0,81	29
1 (Chorzy)	0,82	0,84	0,83	32
Dokładność (Accuracy)			0,82	61
Średnia ważona	0,82	0,82	0,82	61

7. Analiza fragmentów kodu:

Aby zapewnić przejrzystość, modularność oraz odporność na błędy (zgodnie z dobrymi praktykami inżynierii oprogramowania), kod został podzielony na odrębne logiczne etapy. Poniżej przedstawiono analizę najważniejszych mechanizmów użytych w projekcie.

7.1. Wczytywanie i weryfikacja danych (Eksploracja EDA) Pierwszym krokiem w każdym projekcie uczenia maszynowego jest poprawne załadowanie danych i sprawdzenie ich integralności przy użyciu biblioteki Pandas.

Python

```
import pandas as pd

# Wczytanie danych medycznych z pliku CSV
```

```
df = pd.read_csv('heart_disease.csv')

# Sprawdzenie liczby brakujących wartości w poszczególnych kolumnach
print("Liczba brakujących danych:\n", df.isnull().sum())

# Sprawdzenie balansu klas (zdrowi vs chorzy)
print("Rozkład zmiennej docelowej:\n", df['target'].value_counts())
```

Opis działania: Powyższy skrypt automatyzuje proces wstępnej diagnozy zbioru. Zamiast ręcznie przeglądać setki wierszy w Excelu, funkcje `.isnull().sum()` oraz `.value_counts()` błyskawicznie raportują, gdzie znajdują się luki w danych (w naszym przypadku `ca` i `thal`) oraz w jakiej proporcji podzieleni są pacjenci.

7.2. Separacja danych i podział na zbiory (Train/Test Split) Przed rozpoczęciem uczenia, algorytm musi wiedzieć, co jest informacją, a co diagnozą. Konieczne jest też wydzielenie części danych wyłącznie do testów.

Python

```
from sklearn.model_selection import train_test_split

# Oddzielenie cech pacjentów (X) od zmiennej docelowej (y)
X = df.drop('target', axis=1)
y = df['target']

# Podział na zbiór treningowy (80%) i testowy (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Opis działania: Funkcja dzieli dane w proporcji 80/20. Kluczowym parametrem jest tutaj `stratify=y`. Gwarantuje on, że proporcja osób chorych do zdrowych będzie identyczna w zbiorze treningowym (do nauki) i testowym (do egzaminowania modelu), co zapobiega zjawisku wylosowania do testów niemiernodajnej grupy pacjentów.

7.3. Transformacja i czyszczenie danych (Preprocessing) Aby algorytm zadziałał poprawnie, surowe dane medyczne muszą zostać przekształcone do formatu matematycznego za pomocą `ColumnTransformer`.

Python

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Transformacje dla danych liczbowych (np. wiek, ciśnienie)
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Transformacje dla danych kategorycznych (np. typ bólu klatki)
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])
```

Opis działania: Fragment ten definiuje dwie zautomatyzowane ścieżki naprawy danych:

- Dla cech liczbowych braki uzupełniane są **medianą** (odporną na skrajne wartości), a następnie dane są **skalowane** (standaryzacja), co jest niezbędne dla poprawnego działania Regresji Logistycznej.
- Dla cech katagorycznych braki łątane są **dominantą**, a same kategorie są kodowane w systemie zero-jedynkowym (**One-Hot Encoding**), aby model nie nadawał sztucznej ważności nieliczbowym cechom medycznym.

7.4. Budowa zintegrowanego Potoku (Main Pipeline) Wszystkie kroki czyszczenia danych oraz sam model zostały połączone w jeden obiekt.

Python

```
from sklearn.linear_model import LogisticRegression

# Główne narzędzie integrujące przygotowanie danych z algorytmem
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(solver='liblinear', random_state=42))
])
```

Opis działania: Użycie Pipeline to profesjonalna praktyka zapobiegająca wyciekowi danych (data leakage). Gwarantuje, że podczas diagnozowania nowych pacjentów ze zbioru testowego zostaną na nich nałożone dokładnie te same filtry transformacji, co na pacjentów treningowych.

7.5. Optymalizacja hiperparametrów (Grid Search) Zastosowano zautomatyzowane przeszukiwanie przestrzeni parametrów z wykorzystaniem walidacji krzyżowej.

Python

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'classifier__C': [0.01, 0.1, 1, 10, 100],
    'classifier__penalty': ['l1', 'l2']
}

grid_search = GridSearchCV(
    pipeline, param_grid, cv=5, scoring='accuracy'
)
grid_search.fit(X_train, y_train)
```

Opis działania: Kod tworzy "siatkę" możliwych konfiguracji (parametr `C` to siła regularyzacji, a `penalty` to jej typ). Mechanizm uczy i weryfikuje model wielokrotnie dla każdej kombinacji (`cv=5`). Dzięki temu wybór optymalnych parametrów jest wynikiem matematycznego dowodu, a nie zgadywania.

7.6. Generowanie wyników i ewaluacja graficzna Ostatni ważny element to wygenerowanie twardych dowodów skuteczności modelu (tabel i wykresów).

Python

```

from sklearn.metrics import classification_report, RocCurveDisplay
import matplotlib.pyplot as plt

# Pobranie najlepszego modelu z GridSearch i klasyfikacja danych testowych
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

# Generowanie raportu tekstowego
print(classification_report(y_test, y_pred))

# Generowanie i wyświetlenie Krzywej ROC
RocCurveDisplay.from_estimator(best_model, X_test, y_test)
plt.title('Krzywa ROC dla modelu Regresji Logistycznej')
plt.show()

```

Opis działania: Funkcja `classification_report` błyskawicznie porównuje prawdziwe diagnozy z tymi postawionymi przez AI, wyliczając Precyzję, Czułość i F1-Score. Z kolei klasa `RocCurveDisplay` wykorzystuje dane prawdopodobieństwa wygenerowane przez nasz potok (Pipeline) do narysowania wykresu obrazującego zdolność modelu do odróżniania pacjentów zdrowych od chorych (pole AUC).

7.7. Zabezpieczenie punktu wejścia programu (Entry Point) W pliku `main.py` zastosowano konstrukcję chroniącą przepływ aplikacji.

Python

```

def main():
    # Główna logika programu (wywołanie analizy, treningu i wykresów)
    pass

if __name__ == "__main__":
    main()

```

Opis działania: Konstrukcja `if __name__ == "__main__":` sprawia, że skrypt uruchomi uczenie modelu tylko i wyłącznie wtedy, gdy użytkownik świadomie odpali plik `main.py`. Jest to niezbędne przy podziale kodu na wiele plików – zapobiega niekontrolowanemu wykonaniu się kodu podczas importowania funkcji z innych modułów.

8. Podsumowanie

Przeprowadzony projekt pozwolił na pełną realizację postawionych celów, obejmujących analizę danych medycznych oraz budowę skutecznego modelu klasyfikacyjnego opartego na Regresji Logistycznej. Na podstawie uzyskanych wyników oraz przebiegu prac sformułowano następujące wnioski:

Skuteczność modelu w diagnostyce

Zastosowany model osiągnął wysoką stabilność i zadowalającą dokładność na poziomie **82% na zbiorze testowym**. Szczególnie istotnym sukcesem jest uzyskanie wysokiej czułości (Recall) dla grupy osób chorych (**84%**). W kontekście medycznym oznacza to, że

algorytm skutecznie minimalizuje ryzyko przeoczenia choroby u pacjenta, co jest absolutnie kluczowe w systemach wspomagania decyzji lekarskich, gdzie bezpieczeństwo pacjenta jest priorytetem.

Znaczenie przygotowania danych (Preprocessing)

Projekt wykazał, że sukces modelu uczenia maszynowego w dużej mierze zależy od etapu przygotowania danych. Skuteczna obsługa braków danych w kolumnach „ca” i „thal” za pomocą imputacji pozwoliła na zachowanie integralności zbioru. Z kolei zastosowanie standaryzacji cech liczbowych oraz kodowania One-Hot dla cech kategoriowych umożliwiło Regresji Logistycznej sprawiedliwe traktowanie wszystkich parametrów zdrowotnych, niezależnie od ich jednostek czy skali.

Wnioski z optymalizacji hiperparametrów

Przeprowadzone eksperymenty z wykorzystaniem metody GridSearchCV udowodniły, że dobór parametrów ma realny wpływ na jakość predykcji. Wybór kary **L1 (Lasso)** zasugerował, że model potrafi samodzielnie wyłonić najważniejsze wskaźniki chorobowe, co czyni go modelem częściowo "samooczyszczającym się" z szumu informacyjnego. Wysoka wartość parametru **C (100.0)** potwierdziła natomiast, że dane w zbiorze Heart Disease są wiarygodne i pozwalają na silne dopasowanie modelu bez utraty zdolności do generalizacji.

Potencjał i interpretowalność algorytmu

Regresja Logistyczna, mimo swojej relatywnej prostoty w porównaniu do głębokich sieci neuronowych, okazała się narzędziem bardzo wydajnym dla tego typu zadań. Jej główną zaletą jest wysoka interpretowalność – wyniki przedstawione za pomocą Macierzy Pomyłek i Krzywej ROC są jasne i łatwe do zrozumienia dla personelu medycznego. Pozwala to na budowanie zaufania do systemów sztucznej inteligencji w medycynie.